

“DNNs are piecewise sub-differentiable”

Note on Namyar et al., HotNets '24, §4 and footnote 1.

The claim

ReLU, max-pooling and `abs` put kinks in a DNN, so it is not differentiable everywhere. The damage is contained:

1. **Piecewise.** The input space splits into regions; inside one the network is smooth, so ∇ exists. Kinks live only on the boundaries — measure zero.
2. **Sub-differentiable.** On a kink there is no derivative but still a non-empty set of valid slopes. Any one of them is a usable direction.

Differentiable almost everywhere; elsewhere a subgradient stands in. Backprop therefore always returns *some* direction — all the paper’s search loop needs.

The pieces

In a ReLU network each unit is active ($z > 0$) or inactive ($z < 0$). Fixing that pattern fixes a region, on which the network collapses to an affine map:

$$H(x) = W_{\text{eff}} x + b_{\text{eff}} \quad \text{for } x \in R_k$$

The R_k are convex polyhedra, exponentially many, each with its own W_{eff} . Within one the gradient is constant and exact; crossing a boundary flips a unit and jumps it.

The subdifferential

For $\sigma(z) = \text{ReLU}(z) = \max(0, z)$, the slopes at $z = 0$ disagree (0 from the left, 1 from the right), so no tangent exists. The subdifferential replaces the slope with the **set** of slopes of lines touching the graph there and staying below it:

$$\partial\sigma(0) = \{ g : \sigma(z) \geq \sigma(0) + g(z - 0) \quad \forall z \} = [0, 1]$$

Any $g \in [0, 1]$ is a **subgradient**; autodiff just picks one (PyTorch and TensorFlow return 0). Arbitrary but harmless — the point is hit with probability zero, and every element is a legitimate local linear model.

Why the paper needs it

It licenses the chain rule in Eq. (1). The system is a composition, $\mathcal{M}_{adv}(H_n(H_{n-1}(\cdots H_1(x))))$, ascended as

$$x^{(i+1)} \leftarrow x^{(i)} + \alpha \nabla_x \mathcal{M}_{adv}(H(x^{(i)}))$$

Each H_i is differentiated in place and the Jacobians multiplied, so no component need be written in closed form for a solver — the gray-box saving over MetaOpt. Footnote 1 says the same: differentiable layers interspersed with non-differentiable ones, the composite inheriting the structure.

It also cuts the other way (§3.1): white-box tools need piecewise-**linear** activations to encode a DNN as MILP or SMT. Piecewise **sub-differentiability** is far weaker — hence the “complex DNNs that fall outside of this category.”

Two caveats

- **Subgradients are convex analysis.** The global-under-estimator definition needs convex f ; DNNs are not convex. The correct object is the **Clarke subdifferential**, $\partial_C f(x) = \text{conv}\{\lim \nabla f(x_j) : x_j \rightarrow x\}$, which gives the same $[0, 1]$ for ReLU without assuming convexity. The paper says “sub-differentiable” loosely; it leans on Clarke.
- **The chain rule degrades to an inclusion.** In general $\partial(f \circ g) \subseteq \partial f \cdot \partial g$, with equality only under regularity conditions — so composing across many components can give a direction that is not a true subgradient of the composite. This is the approximation every DL framework already makes, and the guarantee is weak regardless: Eq. (1) reaches only a *local* optimum unless $\mathcal{M}_{adv}(H(\cdot))$ is convex, which it is not.